Derin Öğrenme (FET312)

Proje İlerleme Raporu

1) Kapak Sayfası (Proje Meta Verileri)

Ders / Dönem:
FET312 Derin Öğrenme / 2024-2025 Güz Dönemi
Proje Başlığı:
Mobil Cihaz Kullanım Verilerine Dayalı Kullanıcı Davranış Sınıflandırması
Grup İsmi:
NeuralWall
Grup Üyeleri:
ALA BUVEIDANI - alabuveidani@stu.topkapi.edu.tr - 24040301051
GitHub/Repo Linki:
https://github.com/AlaBuveidani/NeuralWall

2) Problem Tanımı & Motivasyon

İş / Bilimsel Soru

Akıllı telefon kullanıcılarının **demografik özellikleri** (yaş, cinsiyet, meslek vb.) ile **kullanım alışkanlıkları** (günlük ekran süresi, sosyal medya kullanımı, oyun / üretkenlik uygulamaları, pil tüketimi vb.) arasındaki ilişkiye bakarak:

- Kullanıcıları **davranış segmentlerine** ayırabilir miyiz?
- "Yoğun kullanıcı", "ortalama kullanıcı", "hafif kullanıcı" gibi davranış kategorilerini **otomatik olarak** tahmin etmek mümkün mü?
- Hangi özellikler, "aşırı kullanım" veya "daha dengeli kullanım" için en belirleyici?

Bu soruları cevaplamak, hem **hedefli pazarlama** hem de **dijital sağlık / dijital esenlik** uygulamaları için değerli içgörüler sağlayabilir.

Görev Türü

- Ana görev: **Sınıflandırma**
  - Kullanıcıyı bir **davranış kategorisine** atama
    - Örnek: *Low / Moderate / High usage* veya veri kümesindeki hazır davranış etiketi varsa onu kullanmak.
- İsteğe bağlı ikinci görev: **Regresyon**
  - Örneğin, **günlük ekran süresi** (dakika/saat) ya da **günlük toplam app kullanım süresi** gibi sürekli bir değeri tahmin etmek.

Bu yüzden proje, şablonda da izin verildiği gibi **hibrit (sınıflandırma + regresyon)** bir yapıda tasarlanabilir.

Hedef Değişken(ler)

- **Sınıflandırma hedefi (y):**
  - Eğer veri kümesinde hazır bir davranış etiketi varsa (ör. *user_behavior_class, usage_segment, user_type* vb.), o sütun **hedef değişken** olarak alınacaktır.
  - Eğer böyle bir sütun yoksa, **kendi etiketimizi** oluşturacağız:
    - Örn. `Daily_Screen_Time` ve toplam uygulama kullanımı üzerinden
      - 0–3 saat: *Low usage*
      - 3–6 saat: *Moderate usage*
      - 6+ saat: *High usage*
- **Regresyon hedefi (opsiyonel):**
  - Sürekli hedef: Günlük toplam ekran süresi, günlük toplam app kullanım süresi, pil tüketimi yüzdesi vb.

Pozitif sınıf (binary senaryo için):

- Örn. `High usage` sınıfını pozitif sınıf olarak tanımlayıp, “yüksek riskli / aşırı kullanım” davranışını tahmin etmeye odaklanabiliriz.

Başarı Kriterleri (nicel hedefler)

Örnek hedefler (raporda sayısal yazmanız gerekiyor):

- Çok sınıflı sınıflandırma için:
  - **Accuracy ≥ 0.80**
  - **Macro F1 Score ≥ 0.78**
- Binary senaryo için:
  - **ROC AUC ≥ 0.85**
  - **PR-AUC** metriklerinin raporlanması (özellikle sınıf dengesizliği varsa).

- Regresyon için (kullanılırsa):
    - **MAE ≤ X saat**
    - **R^2 ≥ 0.70** gibi hedefler.

Bu sınırlar proje başında “hedef”, sonunda da “başardık / başaramadık” şeklinde tartışılacak.

3) Proje Yönetimi

Önemli Noktalar & Zaman Çizelgesi (Gantt tarzı)

- **1-2. Hafta (20 Ekim-2 Kasım):**
    - Veri setinin seçilmesi, Kaggle sayfasının incelenmesi
    - Problem tanımının ve iş sorularının netleştirilmesi
    - Veri keşfi (EDA)
    - Eksik değer analizi, basit istatistikler, sınıf dağılımları
- **3–4. Hafta (3-16 Kasım):** Özellik mühendisliği (feature engineering)
    - Kategorik değişkenlerin kodlanması, sayısal değişkenlerin ölçeklenmesi
    - Basit ML modelleri (Logistic Regression, Random Forest) ile **baseline**
    - PyTorch ile derin öğrenme modellerinin geliştirilmesi (MLP tabanlı modeller)
    - Hiperparametre optimizasyonu (learning rate, layer sayısı, dropout vs.)
- **5. Hafta (17-23 Kasım):**
    - Karşılaştırmalı performans analizi
    - ROC/PR eğrileri, confusion matrix, önemli feature’ların yorumu
    - Nihai proje raporunun yazılması
    - Sunum slaytlarının ve GitHub reposunun hazırlanması

Roller ve Sorumluluklar

- **Öğrenci A:**
    - Veri ön işleme & EDA
    - Baseline klasik ML modelleri (LogReg, RandomForest)
    - PyTorch MLP tabanlı derin model
    - Hiperparametre optimizasyonu
    - Deney tasarımı, sonuçların görselleştirilmesi
    - Rapor yazımı, literatür kısmı

Çıktılar

- **GitHub Repo:**
    - `README.md` - Proje açıklaması ve kullanım kılavuzu

- `requirements.txt` - Gerekli Python paketleri
- `data/` - Veri seti dosyaları
  - mobile_device_usage.csv
- `notebooks/` - Jupyter Notebook dosyaları
  - `FET312_24040301051_NeuralWall_1.ipynb`
- `src/` - Python kaynak kodları (modüler)
- `models/` - Kaydedilen model ağırlıkları (.pth)
- `results/` - Grafikler, metrikler, confusion matrices
- `docs/` - Raporlar ve sunumlar
  - `FET312_24040301051_NeuralWall_ProjectOutline.pdf`

4) İlgili Çalışmalar (Mini Literatür Taraması)

1. **Kaggle Competition / Notebook'lar**
   a. *Behavior-Based Mobile Usage Pattern Analysis* yarışması: aynı veri setiyle kullanıcı davranışını sınıflandırmayı hedefliyor.
   b. Bu yarışmadaki çözümler, çoğunlukla **klasik ML modelleri** (Tree-based, XGBoost, LightGBM) ve basit MLP'ler kullanıyor.
2. **Kaggle Mobile Usage & User Behavior Notebooks**
   a. Veri seti üzerine yazılmış notlar (exploratory notebooks). Bunlarda genellikle:
      i. EDA, korelasyon analizleri
      ii. Basit KNN / SVM / Random Forest modelleri
      iii. Genellikle derin öğrenme odakta değil.
3. **Akademik / blog kaynakları**
   a. Mobil kullanım verileriyle **kullanıcı segmentasyonu** veya **churn prediction** yapan makaleler (ör. telekom operatörü verileri ile churn tahmini, app usage ile bağımlılık analizi vb.).

Bizim Projenin Farkı / Katkısı

- Aynı veri seti üzerinde çoğunlukla **klasik ML** kullanılmış; biz özellikle:
  - **PyTorch tabanlı derin öğrenme** ile tabular veriye yaklaşacağız (MLP, embedding'li modeller vb.).
  - Küçük veri için **ağ mimarisini sade, düzenlileştirmeyi (dropout, weight decay) güçlü** tasarlayacağız.
  - **PR-AUC, ROC-AUC, F1** gibi daha zengin metriklerle değerlendirme yapacağız.
  - Özellik önemini (permutation importance, SHAP/LIME vb.) kullanarak **yorumlanabilirlik** sağlayacağız.

5) Veri Açıklaması ve Yönetimi

Veri Kümesi Açıklaması

- **Adı:** Mobile Device Usage and User Behavior Dataset
- **Kaynak:** Kaggle (veri seti yükleyicisi: Vala Khorasani).
- **Bağlantı:** Kullanıcının paylaştığı Kaggle URL'si.
- **Lisans / Kullanım Hakları:** Kaggle sayfasında belirtilen lisans aynen raporda belirtilecektir (ör. CC tabanlı lisans vb.).

Bu veri kümesi; kullanıcıların **demografik bilgileri** ve **mobil cihaz kullanım alışkanlıkları** üzerinden davranış kalıplarını incelemeyi amaçlamaktadır. Demografi ile app kullanımı, batarya tüketimi ve ekran süresi gibi sinyallerin ortak etkisi üzerinden **kullanıcı davranış sınıflandırması** yapmaya imkân verir.

Şema (Örnek Değişkenler)

Gerçek sütun adlarını Notebook'ta göreceksiniz; raporda aşağıya benzer biçimde yazabilirsiniz:

- Demografik Değişkenler
  - `Age_Group` (kategorik: 18–24, 25–34, …)
  - `Gender` (kategorik)
  - `Occupation` / `StudentStatus` vb.
- Kullanım Özellikleri
  - `Social_Media_Usage` (saat / gün veya yüzde)
  - `Gaming_Usage`, `Productivity_App_Usage`
  - `Battery_Drain_Per_Day` (%), `Charging_Frequency`
  - `Number_of_Apps_Installed`, `Notifications_Per_Day` vb.
- Hedef / Davranış Etiketleri
  - `Usage_Category` veya türettiğimiz `usage_level` (Low/Moderate/High).

Her özellik için:

- **Tip:** (int, float, categorical)
- **Birim:** saat, %, adet vb.
- **Beklenen aralık:** Örn. 0–24 saat, 0–100% vb.

Boyut

Kaggle sayfasına göre veri:

- Tek bir **CSV dosyası**, **~12 kB** boyutlu; bu da sınırlı sayıda satır (örneğin birkaç yüz gözlem) ve ~10–20 sütun olabileceğini gösterir.
- Sınıf dengesini, hedef değişken belirlendikten sonra grafikte göstereceğiz (bar chart ile).

Etik, Gizlilik, Önyargı

- Veri kümesi **anonim**, kişisel kimlik bilgisi içermiyor (isim, telefon numarası, ID vb. yok).
- Yine de:
    - **Demografik değişkenler** üzerinden (yaş, cinsiyet) model kurarken **önyargı** ve adalet sorunları tartışılmalı.
    - "Yüksek kullanım = kötü kullanıcı" gibi önyargılı yorumlardan kaçınılacak; sonuçlar **davranış deseni** olarak yorumlanacak.
    - Veri, etik olarak Kaggle lisansına uygun şekilde sadece akademik amaçla kullanılacak.

6) Yöntemler ve Mimari

Genel Yaklaşım

1. **Veri Ön İşleme**
    a. Eksik değerlerin analizi, gerekirse imputasyon.
    b. Kategorik değişkenler:
        i. Baseline için **One-Hot Encoding**,
        ii. Derin model için **embedding** katmanları (PyTorch Embedding).
    c. Sayısal değişkenler:
        i. Standartlaştırma / Min-Max Scaling.
    d. Hedef değişken:
        i. Çok sınıflı etiketlerin integer kodlanması (`0,1,2`).
2. **Baseline Modeller (Klasik ML)**
    a. Lojistik Regresyon
    b. Random Forest / XGBoost
    c. Bu modellerin sonuçları, derin öğrenme modelinin ne kadar "ek fayda" sağladığını göstermek için referans olacak.
3. **Derin Öğrenme Mimarileri (PyTorch)**

Model 1 – Basit MLP (Tabular NN)

a. Girdi: Tüm sayısal özellikler + kategorikler için embedding'lerin konkatenasyonu
b. Katmanlar:
    i. Input → Dense(64) → ReLU → Dropout(0.3)

        ii. Dense(32) → ReLU → Dropout(0.3)
        iii. Output (softmax / sigmoid)
    c. Kayıp fonksiyonu:
        i. Çok sınıflı için `CrossEntropyLoss`
        ii. Binary için `BCEWithLogitsLoss` (gerekirse class weight ile)

Model 2 – Geliştirilmiş MLP

    d. Aynı yapı + Batch Normalization
    e. Daha küçük / büyük gizli katman sayısı ile hiperparametre araması:
        i. Gizli katman sayısı: 1–3
        ii. Nöron sayısı: 32–128
        iii. Learning rate: 1e-4 – 1e-2 aralığında grid/random search

4. **Eğitim Stratejisi**
    a. Train/Validation/Test ayrımı (örn. 70/15/15, sınıf dengesine dikkat ederek stratified split).
    b. **Early stopping** (val loss veya val F1 takibi).
    c. Model checkpoint (en iyi val skoru için ağırlıkları kaydetme).
5. **Hiperparametre Optimizasyonu**
    a. Basit grid search / random search
    b. Denenecek parametreler:
        i. Learning rate
        ii. Batch size
        iii. Dropout oranı
        iv. Katman sayısı / nöron sayısı

Her grup üyesi, raporda **kendi denediği mimariyi** blok diyagram, katman boyutları, aktivasyon fonksiyonları ve en iyi sonuçlar ile birlikte detaylı anlatacak.

7) Deney Tasarımı

Deney 1 – Klasik ML vs Derin Öğrenme

- **Amaç:**
    - Tabular veri üzerinde klasik ML ile basit MLP'nin performansını karşılaştırmak.
- **Kurulum:**
    - Aynı train/val/test ayrımı
    - Modeller: Logistic Regression, Random Forest, Basit MLP
- **Değerlendirme Metrikleri:**
    - Accuracy, Macro F1, ROC AUC, PR AUC
- **Analiz:**
    - Hangi model hangi metrikte daha iyi?

    - Küçük veri sebebiyle MLP'nin overfit olup olmadığı (train/val loss eğrileri).

Deney 2 – Demografik Özelliklerin Etkisi

- **Amaç:**
    - Demografik değişkenlerin (yaş, cinsiyet vb.) model performansına katkısını görmek.
- **Kurulum:**
    - Model A: Sadece kullanım özellikleri (screen time, app usage, battery)
    - Model B: Kullanım + demografik özellikler
- **Değerlendirme:**
    - Accuracy / F1 karşılaştırması
    - Öznitelik önemi (feature importance / SHAP) üzerinden yorum.
- **Yorum:**
    - Demografik bilgileri kullanmak gerçekten gerekli mi, yoksa kullanım özellikleri yeterli mi?
    - Etik tartışma: Demografik bilgiler çıkarıldığında adalet açısından durum nasıl?

Her deney için raporda:

- Train/val loss grafikleri
- ROC/PR eğrileri
- Confusion matrix ve gerekirse heatmap görselleri appendix kısmına konulacak.

8) Kullanılan Araçlar ve Frameworkler

- **Dil & Ortam**
    - Python 3.x
    - Çalışma ortamı: Google Colab / Jupyter Notebook
- **Kütüphaneler**
    - Veri işleme: `pandas`, `numpy`
    - Görselleştirme: `matplotlib`, `seaborn` (EDA için)
    - ML baseline: `scikit-learn`
    - Derin öğrenme: **`PyTorch`** (zorunlu koşul)
    - İsteğe bağlı: `imblearn` (SMOTE vs.), `shap` (yorumlanabilirlik)
- **Donanım**
    - GPU (Colab / lab makinesi) opsiyonel, küçük veri olduğundan CPU da yeterli.
    - Seed: Tüm grup aynı random seed değerini kullanacak (tekrarlanabilirlik için).

- **Kod Paylaşımı**
  - Tüm kodlar ve Notebook'lar ortak bir GitHub reposunda:
    - `/notebooks`, `/src`, `/figures`, `/reports` klasörleri gibi basit bir yapı.